```
class Lab02 {
    static void Main() {
        Console.WriteLine("Ready");
    }
}
```

Human Computer Interaction

Lab 02

What is a Method?

A method is a named, reusable block of code that performs a task — defined once, called wherever you need it.

Declaring a Method

A method needs a return type, a name, and a parameter list — even if it takes no parameters.

returnType MethodName(parameters)

```
static int Add(int a, int b)
{
    int result = a + b;
    return result;
}
```

Calling a Method

Call a method by its name followed by parentheses. If it returns a value, store it in a variable.

int result = MethodName(arguments);

```
static void Main(string[] args)
{
    int sum = Add(5, 10);
    Console.WriteLine(sum);
}
```

Problem 1:

Rectangle Area

Write a method:

- static double Area(double width, double height)
- that returns width * height.
- Call it from Main with two different rectangles.

Then print:

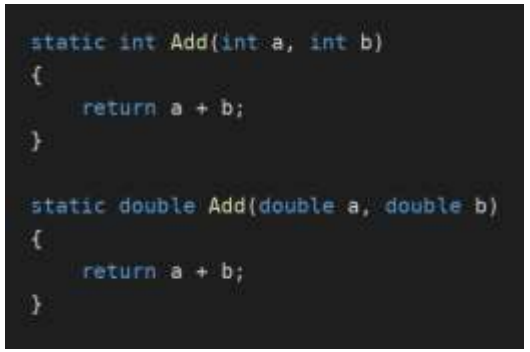
Area of rectangle 1: 24

Area of rectangle 2: 50

Method Overloading

Overloading lets you define several methods with the same name but different parameters — C# picks the right one.

Add(int, int) and Add(double, double)



```
static int Add(int a, int b)
{
    return a + b;
}

static double Add(double a, double b)
{
    return a + b;
}
```

ref vs out Parameters

ref passes an existing variable by reference and must already have a value. out also passes by reference, but the method must assign it.

Use out when a method must return more than one value.

```
static void Square(ref int x)
{
    x = x * x;
}

static void Divide(int a, int b, out int result)
{
    result = a / b;
}
```

Problem 2:

Overloaded Greet Method
Write three overloads of a method called Greet:
Greet() prints “Hello!”
Greet(string name) prints “Hello, name!”
Greet(string name, int age) prints the name and age
Call all three overloads from Main

Arrays as Method Parameters

Arrays are passed by reference, so a method can read — or loop through — the caller's array directly.

int SumArray(int[] numbers)

```
static int SumArray(int[] numbers)
{
    int total = 0;
    foreach (int n in numbers)
        total += n;
    return total;
}
```

Returning Arrays from Methods

A method can build a new array internally and return it, which the caller stores in an array variable.

int[] DoubleValues(int[] numbers)

```
static int[] DoubleValues(int[] numbers)
{
    int[] result = new int[numbers.Length];
    for (int i = 0; i < numbers.Length; i++)
        result[i] = numbers[i] * 2;
    return result;
}
```

Tuples:

A **tuple** is a data structure that lets you **group multiple values together into a single object.**

Unlike an array, a tuple can contain values of **different types**.

```
var person = ("Ahmed", 20);
Console.WriteLine($"Person's name is {person.Item1}," +
    $" Age is {person.Item2}");

(string name, int age) person2 = ("Ali", 19);
Console.WriteLine($"Person's name is {person2.name}, " +
    $"Age is {person2.age}");
```

Methods can return tuples as well

```
1 reference
static (int x, int y) OperateOnNumbers (int a, int b)
{
    return (a * b, a + b);
}
```

```
var item = OperateOnNumbers(5, 6);
Console.WriteLine(item.x);
Console.WriteLine(item.y);
```

Problem 3:

Array Statistics Methods
Write a C# program that reads 6 scores from the user into an int array.
Write three methods that each take the array as a parameter:
GetAverage(int[] scores) returns the average as a double.
GetMax(int[] scores) returns the highest score.
CountAbove(int[] scores, int threshold) counts scores above a given value.
Call all three from Main and print:

- The average score.
- The highest score.
- How many scores were above 70.

What is Recursion?

Recursion is when a method calls itself, moving toward a base case that stops the calls

Recursion Example: Factorial

Factorial(n) calls Factorial(n - 1) until it reaches the base case n <= 1.

Base case: n <= 1 → return 1.

```
static int Factorial(int n)
{
    if (n <= 1)
        return 1;
    return n * Factorial(n - 1);
}
```

Problem 4:

Write a recursive method:
SumArray(int[] numbers, int index) sums the array recursively.
Base case: when index == numbers.Length, return 0.
Otherwise, return numbers[index] + SumArray(numbers, index + 1).
In Main:

- Fill an array of 5 numbers from the user.
- Call SumArray(numbers, 0) and print the result.
- Bonus: write a similar recursive method to find the maximum.

String operations

Operation	Example	Result
Length	text.Length	11
Access character	text[0]	'H'
Uppercase	text.ToUpper()	"HELLO WORLD"
Lowercase	text.ToLower()	"hello world"
Contains	text.Contains("World")	true
StartsWith	text.StartsWith("Hello")	true
EndsWith	text.EndsWith("World")	true
Replace	text.Replace("World", "C#")	"Hello C#"
Trim	text.Trim()	Removes leading/trailing spaces
IndexOf	text.IndexOf("World")	6
Substring	text.Substring(0, 5)	"Hello"

Problem 5:

Write a C# program that asks the user to enter a sentence.
The program should:
Display the number of characters in the sentence.
Display the sentence in uppercase.
Display the sentence in lowercase.
Ask the user to enter a word and determine whether that word exists in the sentence.
Display the first character and the last character of the sentence.
Display the sentence with all spaces replaced by -.

Classes:

A class is a blueprint for creating objects.
It defines:

- Data that an object contains
- Behavior that an object can perform
- The structure and capabilities of a particular type

A class can contain:

- **Fields / Properties** → represent data
- **Methods** → represent behavior

Defining a class:

```
2 references
public class Cat
{
    1 reference
    public string Name { get; set; }
    1 reference
    public int Age { get; set; }

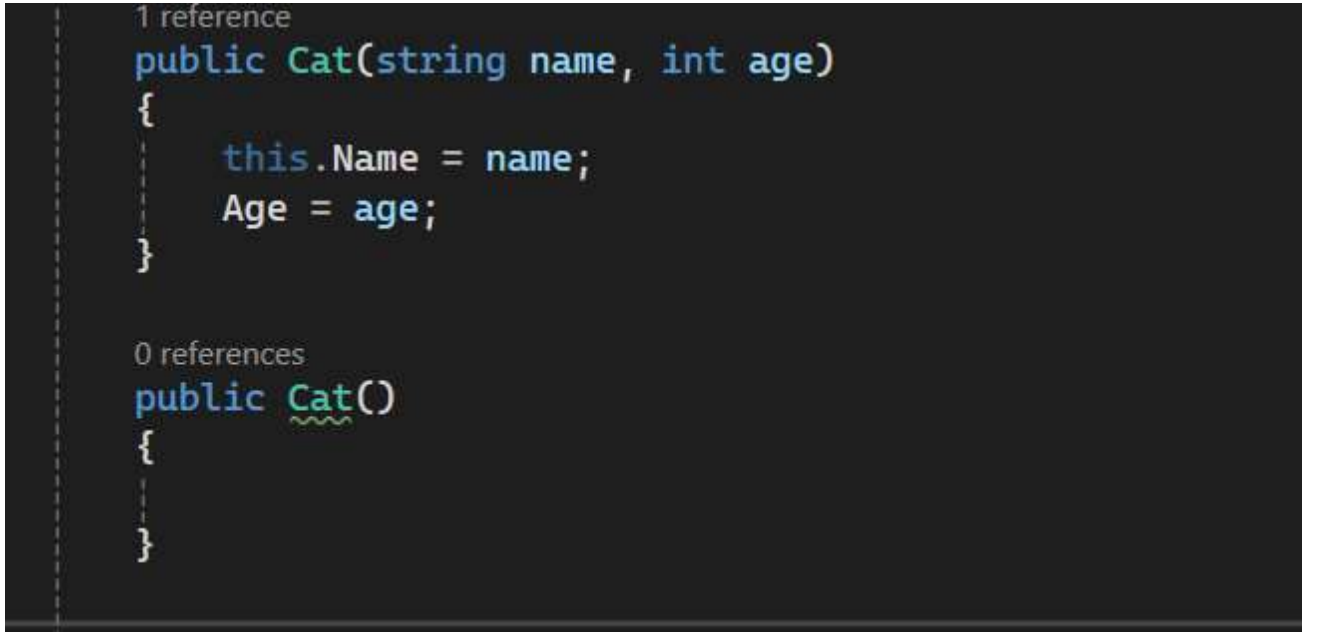

    1 reference
    public void MakeASound()
    {
        Console.WriteLine("Meow");
    }

}
```

```
Cat c = new Cat();
c.Name = "garfield";
c.Age = 1;
c.MakeASound();
```

Constructor:

A constructor is a special method used when an object is created.

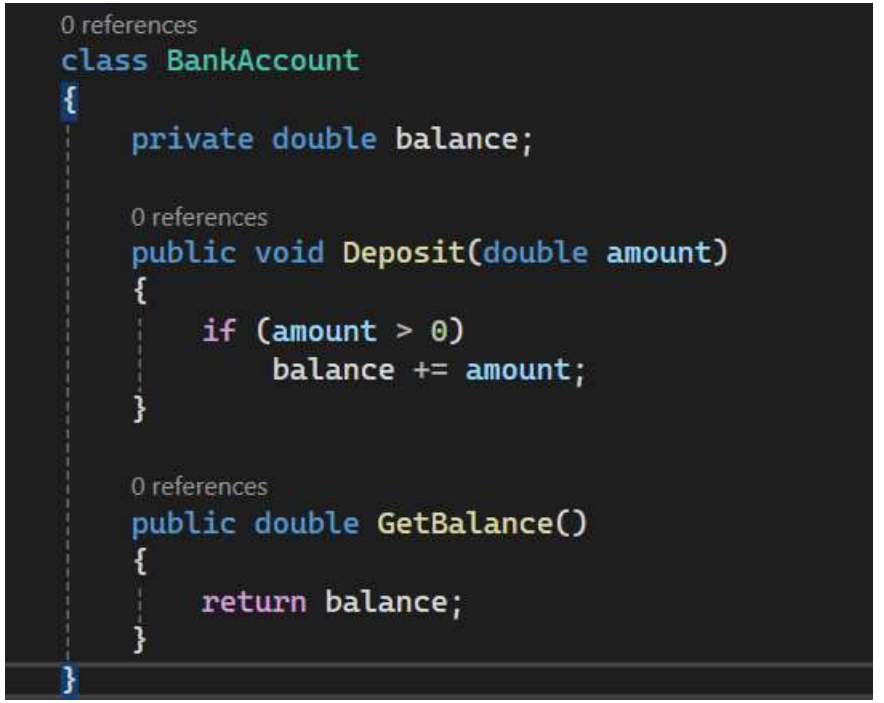


Constructor vs method

Constructor	Method
Same name as the class	Has its own name
No return type	Has a return type or `void`
Runs when object is created	Called explicitly
Initializes the object	Performs an operation

Access modifiers:

Public:
Accessible from outside the class.
public string Name;
Private:
Accessible only inside the class.
private double balance;



Problem 6:

Create a Student class with:
Name
ID
Array of grades (Size 3)
Add the following methods:
CalculateAverage()
GetHighestGrade()
GetLowestGrade()
IsPassed()
PrintInfo()